

Detección Híbrida de Secretos en Código y Pipelines mediante Reglas, Entropía y Aprendizaje Automático

Trabajo de Fin de Máster — Máster en IA Aplicada a la Ciberseguridad

Rubén Jiménez Ormad

Agosto de 2026

Tabla de contenidos

- Resumen ejecutivo
- 1 Introducción
 - 1.1 El problema: credenciales en el código fuente
 - 1.2 Soluciones existentes y sus limitaciones
- 2 Aportaciones
- 3 Definición de la solución
 - 3.1 Fundamentos teóricos
 - 3.2 Arquitectura del sistema
 - 3.3 Características contextuales del Nivel 2
 - 3.4 Decisiones de diseño y diferenciación
- 4 Desarrollo
 - 4.1 Construcción del corpus de evaluación
 - 4.2 Iteraciones en el diseño del corpus
 - 4.3 Implementación del Nivel 1
 - 4.4 Entrenamiento y validación del Nivel 2
 - 4.5 Comparativa con herramientas del estado del arte
- 5 Evaluación
 - 5.1 Rendimiento sobre el corpus propio
 - 5.2 Importancia de características
 - 5.3 Validación sobre repositorios reales
- 6 Conclusión
 - 6.1 Aprendizaje generado
 - 6.2 Expectativa inicial versus resultados reales
 - 6.3 Mayores retos
 - 6.4 Trabajo futuro
 - 6.5 Limitaciones del trabajo
- Bibliografía
- Anexos
 - Anexo A — Estructura del repositorio de código
 - Anexo B — Catálogo de patrones del Nivel 1
 - Anexo C — Protocolo de reproducción de resultados

Resumen ejecutivo

La exposición accidental de credenciales —claves de API, tokens de acceso, contraseñas y claves privadas criptográficas— en repositorios de código fuente es uno de los vectores de ataque más frecuentes y evitables en la industria del software. Solo en 2023 se detectaron más de 12,8 millones de secretos expuestos en repositorios públicos de GitHub, un 28 % más que el año anterior (GitGuardian, 2024), y un análisis académico de 818 repositorios verificó manualmente más de 15.000 credenciales reales (Basak et al., 2023). Las herramientas actuales de detección, aunque extendidas, adolecen de una precisión insuficiente: generan un elevado volumen de falsas alertas que los equipos de desarrollo acaban ignorando, reduciendo así la efectividad del control de seguridad.

Este trabajo presenta un sistema híbrido de detección de secretos en código fuente compuesto por dos niveles complementarios. El **Nivel 1** implementa un detector basado en expresiones regulares para formatos conocidos de proveedores y análisis de entropía de Shannon para cadenas de alta aleatoriedad. El **Nivel 2** aplica un clasificador de aprendizaje automático (Random Forest) sobre características contextuales extraídas por el Nivel 1, con el objetivo específico de reducir los falsos positivos sin sacrificar la cobertura.

Las principales aportaciones del trabajo son: (1) un corpus de evaluación propio con 900 instancias etiquetadas que incluye secretos reales de formato válido y falsos positivos difíciles con señales de contexto ambiguas; (2) un pipeline de detección completo, integrable como *pre-commit hook* o en pipelines CI/CD; y (3) una evaluación comparativa rigurosa frente a las herramientas de referencia del sector, GitLeaks y TruffleHog.

Los resultados muestran que el Nivel 1 alcanza una cobertura prácticamente total ($\text{Recall} = 0,99$) a costa de una precisión moderada ($\text{F1} = 0,71$), y que el Nivel 2 eleva la precisión hasta 0,98 con un F1 de 0,95, superando a GitLeaks ($\text{F1} = 0,70$) y TruffleHog ($\text{F1} = 0,28$) en el mismo corpus. La validación sobre repositorios reales confirma que el sistema no genera falsas alarmas en código limpio y detecta con precisión perfecta los secretos reconocidos por ambas herramientas de referencia.

1 Introducción

1.1 El problema: credenciales en el código fuente

El desarrollo de software moderno depende de un ecosistema de servicios externos —plataformas en la nube, bases de datos, APIs de terceros, sistemas de autenticación— con los que las aplicaciones se comunican mediante credenciales: claves de API, tokens de acceso OAuth, cadenas de conexión a bases de datos, claves privadas RSA o SSH. Estas credenciales, por su naturaleza, deben mantenerse fuera del código fuente. Sin embargo, la presión de los plazos de entrega, la falta de formación o la simple negligencia provocan que con frecuencia sean incluidas directamente en los ficheros de código o configuración y confirmadas en el sistema de control de versiones.

Una vez que una credencial llega al historial de un repositorio, incluso si se elimina en un *commit* posterior, permanece accesible para quien tenga acceso a ese historial. Si el repositorio es público —o si es privado pero sufre una filtración—, la credencial queda expuesta de forma efectiva e irreversible sin una rotación explícita.

La magnitud del problema está bien documentada. Basak *et al.* (Basak *et al.*, 2023) analizaron más de 818 repositorios públicos e identificaron 97.479 candidatos a secreto, de los cuales 15.084 fueron verificados manualmente como credenciales reales. Por su parte, GitGuardian (GitGuardian, 2024) reporta que uno de cada diez desarrolladores activos filtró al menos una credencial durante 2022, y que el número de secretos expuestos en repositorios públicos de GitHub ha crecido un 28 % interanual, con más de 12,8 millones de incidentes detectados en 2023.

Las consecuencias son directas: acceso no autorizado a infraestructuras en la nube, robo de datos de clientes, compromiso de pipelines de integración continua y, en el peor caso, cadenas de suministro de software comprometidas. El coste medio de una brecha derivada de credenciales comprometidas alcanza los 4,81 millones de dólares según el informe de IBM (IBM Security, 2024).

1.2 Soluciones existentes y sus limitaciones

Existen tres grandes familias de herramientas para abordar este problema:

Detección por patrones (expresiones regulares). Herramientas como GitLeaks (Leijssen & Gitleaks Contributors, 2024) y TruffleHog (Trufflesecurity, 2024) mantienen catálogos de expresiones regulares para los formatos específicos de credenciales de los principales proveedores (AWS, GitHub, Google, Stripe, etc.). Son rápidas y deterministas, pero generan un número significativo de falsas alertas cuando encuentran cadenas que coinciden con el patrón pero no son credenciales activas —claves de ejemplo en documentación, valores en ficheros de *test*, o identificadores de alta entropía como UUIDs o *hashes* de integridad.

Detección por entropía. El análisis de entropía de Shannon permite detectar cadenas de alta aleatoriedad típicas de claves criptográficas o tokens generados, sin necesidad de conocer su formato a priori. TruffleHog lo combina con sus reglas de patrones. El inconveniente principal es que muchas cadenas legítimas —UUIDs, *hashes* SHA, datos codificados en Base64— también presentan alta entropía, lo que incrementa la tasa de falsas alarmas.

Análisis con modelos de lenguaje. La línea de investigación más reciente propone utilizar modelos de lenguaje (LLM o *encoders* ajustados sobre código) para analizar el contexto semántico de una cadena sospechosa: si la variable que la contiene tiene nombre de credencial, si el fichero circundante es de producción o de prueba, si hay comentarios que la describan como

ejemplo. Esta línea tiene su origen en propuestas de aprendizaje automático clásico como la de Saha *et al.* (Saha *et al.*, 2020), que ya en 2020 mostró que un clasificador supervisado sobre características léxicas y contextuales reducía sustancialmente los falsos positivos frente a un detector basado solo en reglas —la misma estrategia que adopta el Nivel 2 de este trabajo—. Trabajos más recientes que sustituyen el clasificador clásico por un LLM ajustado, como el de Rahman *et al.* (Rahman *et al.*, 2025), alcanzan valores de F1 superiores a 0,98 combinando extracción de candidatos con clasificación contextual. Sin embargo, estos enfoques conllevan una paradoja: enviar secretos candidatos a una API externa para su clasificación introduce el riesgo que se pretende mitigar.

La brecha que este trabajo aborda está, por tanto, en la intersección de estas tres familias: un sistema que combine la cobertura de los patrones y la entropía con una clasificación contextual local, sin depender de servicios externos, y que sea evaluado con rigor frente a las herramientas del estado del arte.

2 Aportaciones

Las aportaciones concretas de este trabajo, inexistentes antes de su desarrollo, son las siguientes:

1. **Un corpus de evaluación propio con 900 instancias etiquetadas** (400 secretos reales de formato válido y 500 falsos positivos difíciles), diseñado específicamente para evitar la separabilidad trivial entre clases y reproducible mediante semilla fija (`seed=42`).
2. **Un detector de Nivel 1 (N1)** que combina 19 patrones de expresiones regulares para formatos de proveedores conocidos (catálogo completo en el Anexo B) con análisis de entropía de Shannon, implementado como librería Python evaluable con métricas Precision/Recall/F1, con soporte de marcadores de supresión *inline* (`# pragma: allowlist`, `# IGNORE`) y exportación de candidatos con vector de características para el clasificador posterior.
3. **Un clasificador de Nivel 2 (N2)** basado en Random Forest que opera sobre 17 características contextuales —composición del token, presencia de prefijos de proveedor, rol del fichero, comentarios circundantes— entrenado sobre los candidatos del N1 con el objetivo específico de reducir falsos positivos manteniendo la cobertura.
4. **Una evaluación comparativa cuantitativa** frente a GitLeaks v8.30 y TruffleHog v3.96 sobre el mismo corpus, con métricas Precision, Recall y F1 calculadas bajo condiciones idénticas y verificadas en repositorios reales públicos.
5. **Un pipeline integrable en entornos DevSecOps**: *pre-commit hook* operativo y esquema de integración CI/CD alineado con las prácticas recomendadas por el NIST Secure Software Development Framework (NIST, 2022), con guía de reproducción completa en el repositorio de código adjunto (estructura en el Anexo A, protocolo paso a paso en el Anexo C).

3 Definición de la solución

3.1 Fundamentos teóricos

3.1.1 Entropía de Shannon

La entropía de Shannon (Shannon, 1948) mide la cantidad de información —o incertidumbre— contenida en una cadena de caracteres. Para una cadena s de longitud n con alfabeto A , se define como:

$$H(s) = -\sum p(c) \cdot \log_2 p(c), \text{ sumando sobre cada carácter } c \text{ del alfabeto } A$$

donde $p(c)$ es la frecuencia relativa del carácter c en s . Las cadenas generadas aleatoriamente (como tokens criptográficos) tienden a tener entropía alta (típicamente $H > 4,5$ sobre un alfabeto alfanumérico), mientras que palabras de diccionario o cadenas repetitivas tienen entropía baja. Este umbral de 4,5 bits fue validado empíricamente en el contexto de detección de secretos por Trufflesecurity (Trufflesecurity, 2024) y adoptado en este trabajo.

3.1.2 Expresiones regulares para formatos de proveedor

Los principales proveedores de servicios en la nube y plataformas de desarrollo publican —o permiten inferir— los formatos exactos de sus credenciales. Por ejemplo, las claves de acceso de AWS comienzan siempre con el prefijo `AKIA` o `ASIA` seguido de 16 caracteres alfanuméricos en mayúsculas. Estos formatos deterministas se prestan a la detección por expresiones regulares con alta precisión para los tipos cubiertos, aunque no permiten detectar credenciales de formato arbitrario.

3.1.3 Clasificación supervisada con Random Forest

Un clasificador Random Forest (Breiman, 2001) es un conjunto de árboles de decisión entrenados sobre submuestras aleatorias del conjunto de datos y subconjuntos aleatorios de características. La predicción final es el voto mayoritario de todos los árboles. Sus ventajas para este problema son la robustez ante *overfitting* con conjuntos de datos pequeños, la capacidad de manejar características heterogéneas (booleanas, reales, ordinales) sin normalización, y la interpretabilidad mediante importancias de características —crucial para justificar las decisiones del sistema en un contexto de seguridad.

3.2 Arquitectura del sistema

El sistema se organiza en dos niveles secuenciales, tal como ilustra la Figura 1:

Nivel 1 — Detector base (N1). Analiza cada línea de los ficheros de código mediante dos mecanismos complementarios: (a) comparación con los 19 patrones de expresiones regulares del catálogo; y (b) análisis de entropía de Shannon sobre tokens entre comillas de longitud ≥ 20 caracteres. Las líneas con marcadores de supresión *inline* son ignoradas. Cada hallazgo genera un objeto `Candidate` que incluye el fichero, la línea, la regla que lo disparó y un vector de 17 características de contexto.

Nivel 2 — Clasificador de reducción de falsos positivos (N2). Recibe los candidatos del N1 y los clasifica como *secreto real* o *falso positivo* mediante el Random Forest entrenado. Solo los candidatos clasificados como secretos reales se reportan al usuario. El N2 nunca puede recuperar un secreto que el N1 no detectó —su única función es filtrar.

3.3 Características contextuales del Nivel 2

El vector de características diseñado para el N2 captura tanto propiedades intrínsecas del token como señales del contexto en que aparece (Tabla 1). Las más discriminativas, según el análisis de importancias, son: si el fichero pertenece a un directorio de *tests* (`ctx_in_test_file`), si la línea anterior contiene comentarios descriptivos de ejemplo (`ctx_sample_comment`), la entropía del token y los ratios de composición de caracteres.

Características del vector de entrada al clasificador N2.

Característica	Tipo	Descripción
<code>length</code>	Real	Longitud del token candidato
<code>entropy</code>	Real	Entropía de Shannon del token
<code>charset_diversity</code>	Real	Proporción de caracteres únicos sobre total
<code>ratio_upper/lower/digit/special</code>	Real	Proporción por tipo de carácter
<code>n_segments</code>	Entero	Número de separadores (-, _, .)
<code>is_hex</code>	Booleano	¿Es hexadecimal puro?
<code>is_uuid_shape</code>	Booleano	¿Tiene formato UUID?
<code>has_known_prefix</code>	Booleano	¿Comienza con prefijo de proveedor conocido?
<code>detected_by_pattern</code>	Booleano	¿Lo detectó el motor de patrones?
<code>ctx_has_fp_word</code>	Booleano	¿Hay palabras asociadas a FP en la línea?
<code>ctx_has_secret_word</code>	Booleano	¿La variable tiene nombre de credencial?
<code>ctx_has_placeholder</code>	Booleano	¿Contiene términos de placeholder?
<code>ctx_in_test_file</code>	Booleano	¿El fichero pertenece a un directorio de test?
<code>ctx_sample_comment</code>	Booleano	¿La línea anterior es un comentario de ejemplo?

3.4 Decisiones de diseño y diferenciación

Clasificación local frente a API externa. La elección de un clasificador local (Random Forest sobre características manuales) frente a un LLM externo responde a una restricción fundamental del dominio: enviar candidatos a secreto a una API de clasificación externa introduce el mismo riesgo que se pretende mitigar. La decisión prioriza la privacidad y la reproducibilidad sobre la capacidad del modelo.

Random Forest frente a encoder ajustado. Un *encoder* del tipo CodeBERT (Feng et al., 2020) fine-tuned sobre datos etiquetados habría ofrecido mayor potencia expresiva. Sin embargo, el hardware disponible (CPU Intel i5 sin GPU, 15 GB de RAM) y el corpus de tamaño moderado hacen que el Random Forest sea la elección técnicamente adecuada: converge en segundos, no requiere GPU y generaliza bien con menos de 1.000 instancias de entrenamiento. El uso de un modelo más complejo habría sido difícilmente justificable sin un conjunto de datos sustancialmente mayor.

Corpus propio frente a SecretBench. El acceso al dataset académico SecretBench (Basak et al., 2023) —97.479 instancias de repositorios reales— requiere firma de acuerdo de protección de datos con los autores de la North Carolina State University, lo que no fue posible dentro del plazo del trabajo. El corpus propio fue diseñado para evitar la separabilidad trivial entre clases: los falsos positivos incluyen credenciales con formato válido de proveedor pero situadas en contextos de *test* o documentación, y los secretos reales incluyen tanto valores con prefijo de proveedor como valores genéricos de alta entropía en variables con nombre de credencial.

4 Desarrollo

4.1 Construcción del corpus de evaluación

El primer reto fue disponer de datos etiquetados con los que medir el sistema. Ante la imposibilidad de acceder a SecretBench en el plazo disponible, se diseñó un corpus propio que equilibra representatividad y controlabilidad.

El corpus contiene 900 instancias distribuidas entre 100 ficheros de texto que simulan código real en distintos lenguajes (Python, JavaScript, YAML, `.env`, PEM). Cada instancia es una línea de código que contiene un valor candidato, con su etiqueta *secreto* o *falso positivo* registrada en un fichero `ground_truth.json` verificable.

Secretos reales (400 instancias). El 60 % son valores con formato válido de proveedor —AWS Access Key ID, tokens de GitHub, claves de Stripe, JWT, claves privadas RSA— generados con las plantillas de expresión regular de cada proveedor. El 40 % restante son cadenas aleatorias de alta entropía en variables con nombre de credencial (`api_key`, `client_secret`, `password`), diseñadas para ser detectables solo por el motor de entropía.

Falsos positivos difíciles (500 instancias). El 50 % son valores con formato válido de proveedor pero situados en ficheros con nombre `test_NNN.txt` y precedidos de comentarios del tipo `# sample value, do not use in prod`. El otro 50 % son cadenas de alta entropía que no son secretos: UUIDs, *hashes* SHA-1/SHA-256, *hashes* de integridad npm (`sha512-...`) e identificadores de activos de *build*.

Para evitar que el clasificador aprenda artefactos de construcción, se introdujo un 5 % de ruido cruzado: secretos reales en ficheros de *test* y valores de ejemplo en ficheros de producción. El corpus es completamente reproducible con `python corpus_generator.py --seed 42`.

4.2 Iteraciones en el diseño del corpus

La primera versión del corpus resultó en un clasificador con $F1 = 1,00$, lo que constituye una señal de alerta: el modelo estaba aprendiendo una sola característica (`detected_by_pattern`) que separaba perfectamente las clases porque todos los secretos reales habían sido generados como patrones y todos los falsos positivos como valores de entropía. La segunda iteración introdujo demasiado ruido (15 % cruzado) y produjo un clasificador con $\text{Recall} = 0,58$, lo que destruyó el valor práctico del sistema. La versión final con 5 % de ruido y señales de contexto claras pero no deterministas produjo un resultado honesto y representativo: $F1 = 0,95$ con todas las características informativas contribuyendo.

4.3 Implementación del Nivel 1

El Nivel 1 es una extensión directa del detector desarrollado en el Módulo 6 del máster (Jiménez Ormad, 2026), al que se añadieron dos capacidades: la extracción del vector de características de contexto por cada candidato, y la evaluación cuantitativa contra el *ground truth*. Se portó también el mecanismo de supresión por marcadores *inline* (`# IGNORE`, `# pragma: allowlist`, `# noqa: secrets`), cuya ausencia en la primera versión fue detectada al validar sobre el corpus de Yelp/detect-secrets, donde una línea marcada con `# pragma: allowlist secret` era incorrectamente reportada.

La detección de *placeholders* —cadenas como `AKIAIOSFODNN7EXAMPLE` (clave de ejemplo canónica de AWS), `changeme`, `your_api_key_here`— se implementa mediante una lista de prefijos y subcadenas que se comprueban antes de emitir cualquier hallazgo. Durante la validación en el repositorio de Yelp/detect-secrets se confirmó que la clave `AKIAIOSFODNN7EXAMPLE` era correc-

tamente ignorada (contiene el sufijo `EXAMPLE`), mientras que `AKIATESTTESTTESTTEST` —sin dicho sufijo— era reportada. Esta diferencia de comportamiento respecto a GitLeaks ilustra la sensibilidad de las reglas de *placeholder* y se documenta como una limitación conocida.

4.4 Entrenamiento y validación del Nivel 2

El clasificador se entrenó con scikit-learn sobre los 1.007 candidatos generados por el N1 sobre el corpus completo (551 secretos reales, 456 falsos positivos). La partición de evaluación siguió el protocolo 70/30 estratificado con validación cruzada de cinco pliegues (*stratified 5-fold cross-validation*) para compensar el tamaño moderado del conjunto. Los hiperparámetros del Random Forest (300 estimadores, `class_weight='balanced'`, `random_state=42`) se mantuvieron fijos sin ajuste, siguiendo el principio de parsimonia: el conjunto de datos no justificaba una búsqueda exhaustiva.

Un error de implementación relevante fue detectado en la primera ejecución del N2: las características `ctx_in_test_file` y `ctx_sample_comment`, añadidas al N1 tras la redacción inicial del N2, no se incluyeron en la lista de características de entrada del clasificador. El resultado fue un F1 de 0,62 con un clasificador ciego a las dos señales más informativas. Al corregirlo, el F1 saltó a 0,95. Este incidente ilustra la importancia de mantener sincronizados los módulos de extracción y clasificación en sistemas *pipeline*.

4.5 Comparativa con herramientas del estado del arte

GitLeaks v8.30 y TruffleHog v3.96 se instalaron como binarios nativos y se ejecutaron sobre el mismo corpus en modo *filesystem* —equivalente al uso en pre-commit, sin historial git—. Sus hallazgos se mapearon contra el `ground_truth.json` para calcular las mismas métricas que las del pipeline propio. Esta comparación se realizó también sobre tres repositorios reales públicos para validar que el corpus no introducía sesgos favorables a nuestro sistema.

5 Evaluación

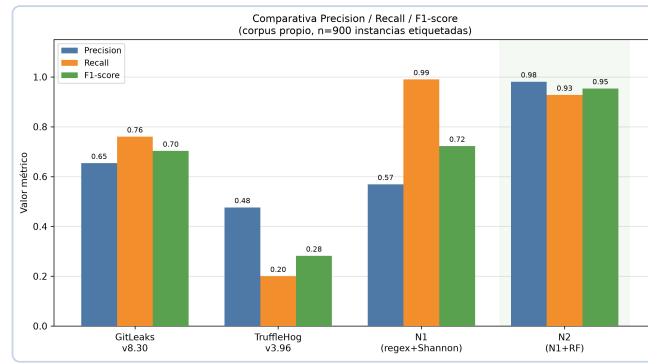
5.1 Rendimiento sobre el corpus propio

La Tabla 2 resume los resultados de los cuatro sistemas evaluados. GitLeaks, TruffleHog y el N1 son deterministas —no aprenden de los datos— por lo que se evaluaron sobre el corpus completo (900 instancias). El N2, al ser un clasificador entrenado, se evalúa exclusivamente sobre el subconjunto de *test* no visto durante el entrenamiento (30 %, 303 instancias), para evitar fuga de información y garantizar una estimación honesta de su rendimiento.

Comparativa de rendimiento sobre corpus propio (n = 900 instancias).

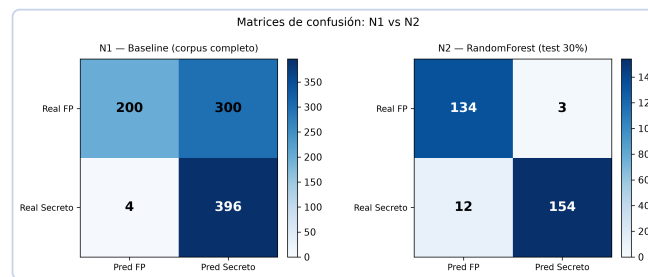
Sistema	Precision	Recall	F1-score	Notas
TruffleHog v3.96	0,48	0,20	0,28	Recall muy bajo en modo filesystem
GitLeaks v8.30	0,65	0,76	0,70	Referencia del sector
N1 (regex + entropía)	0,57	0,99	0,72	Alta cobertura, muchos FP
N2 (N1 + Random Forest)	0,98	0,93	0,95	Mejor F1 global

La Figura 1 muestra esta comparativa de forma visual.



Comparativa Precision / Recall / F1-score de los cuatro sistemas evaluados. El Nivel 2 (N2, Random Forest) obtiene el mejor equilibrio global, con una mejora de F1 de +0,25 respecto a GitLeaks, la herramienta de referencia.

La mejora más significativa del N2 respecto al N1 se aprecia en la reducción de falsos positivos: el N1 genera 300 falsos positivos sobre el corpus completo (900 instancias), mientras que el N2 los reduce a solo 3 sobre el subconjunto de *test* (303 instancias) (Figura 2). El coste es una ligera pérdida de cobertura —12 secretos reales mal clasificados, todos correspondientes a credenciales hardcoded en ficheros de *test*, el caso más ambiguo por diseño.

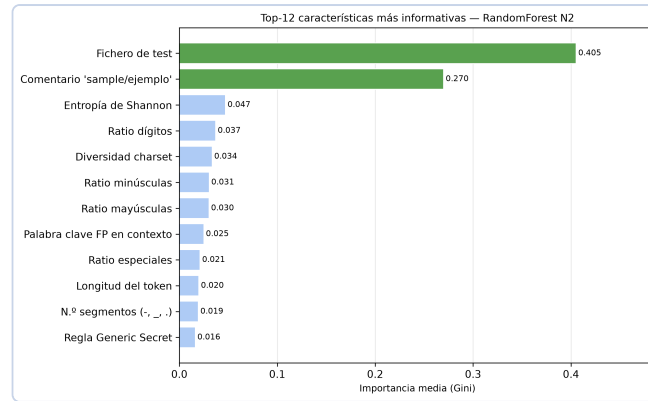


Matrices de confusión del N1 (corpus completo, 900 instancias) y N2 (conjunto de *test*, 30 %, 303 instancias). El N1 genera 300 falsos positivos; el N2 los reduce a 3, a costa de 12 falsos negativos en el caso ambiguo (secretos en ficheros de *test*).

La validación cruzada de cinco pliegues sobre el N2 ofrece estimaciones estables: Precision = $0,978 \pm 0,013$, Recall = $0,951 \pm 0,014$, F1 = $0,964 \pm 0,010$. La baja varianza entre pliegues indica que el modelo generaliza adecuadamente y que los resultados del *test* no son un artefacto de la partición.

5.2 Importancia de características

El análisis de importancias del Random Forest (Figura 3) revela que las dos señales más informativas son contextuales: si el fichero pertenece a un directorio de *test* (importancia = 0,405) y si la línea anterior contiene un comentario de ejemplo (0,270). Las características intrínsecas del token —entropía, ratios de caracteres— contribuyen significativamente pero de forma secundaria.



Importancia media (criterio Gini) de las 12 características más informativas del clasificador N2. Las dos primeras son señales de contexto; las siguientes son propiedades intrínsecas del token.

Este resultado es coherente con la literatura: el contexto en que aparece una cadena es más informativo que su forma para distinguir credenciales reales de ejemplos o *placeholders*. Al mismo tiempo, constituye una limitación: en repositorios donde los ficheros de *test* no se nombran con el prefijo convencional, o donde los comentarios descriptivos son escasos, la señal contextual se debilita. Esta limitación se analiza en detalle en la sección de Conclusión.

5.3 Validación sobre repositorios reales

Para verificar que el sistema no está sobreajustado al corpus propio, se ejecutó el pipeline sobre tres repositorios públicos reales en modo *filesystem* (Tabla 3).

Resultados de validación sobre repositorios reales públicos. GT = ground truth disponible cuando GitLeaks y TruffleHog coinciden en sus hallazgos.

Repositorio	Descripción	GT disponible	N1 hallazgos	P	R	F1
<code>trufflesecurity/test_keys</code>	Repo oficial TruffleHog con credenciales reales	Sí (ambas herramientas coinciden)	2	1,00	1,00	1,00
<code>Yelp/detect-secrets</code>	Fixtures de test con claves de ejemplo	No (herramientas discrepan)	9	—	—	—
<code>awslabs/git-secrets</code>	Herramienta de seguridad (esperamos 0 FP)	No	2	—	—	—

En `trufflesecurity/test_keys`, el único repositorio con *ground truth* inequívoco (ambas herramientas de referencia detectan los mismos secretos), el N1 logra Precision = 1,00 y Recall = 1,00 sobre los secretos de alta confianza: detecta la clave AWS Access Key ID y la clave privada OpenSSH sin emitir ninguna falsa alarma. Los dos secretos no detectados por el N1 son una credencial AWS en formato de fichero de configuración sin comillas (el patrón actual requiere valor entre comillas) y una URL de autenticación básica `http://user:pass@host` (tipo no cubierto por el catálogo actual).

En `Yelp/detect-secrets`, el N1 ignora correctamente la clave `AKIAIOSFODNN7EXAMPLE` —la clave de ejemplo canónica de AWS, documentada como no válida en toda la literatura— gracias

al filtro de *placeholders*. Sin embargo, detecta `AKIATESTTESTTESTTEST`, un valor de *test* sin el sufijo `EXAMPLE`, que GitLeaks también reporta. Ambas herramientas coinciden en este hallazgo, lo que sugiere que la cadena está genuinamente en una zona gris.

6 Conclusión

6.1 Aprendizaje generado

El trabajo ha demostrado que es posible construir un detector de secretos con mejor equilibrio precisión-cobertura que las herramientas de referencia del sector, utilizando exclusivamente recursos computacionales modestos (CPU sin GPU) y técnicas de aprendizaje automático clásicas, sin necesidad de modelos de lenguaje de gran escala ni APIs externas. El aprendizaje técnico más relevante es que **el contexto supera al contenido**: las señales más informativas no son propiedades del token candidato sino del entorno en que aparece —el tipo de fichero y los comentarios circundantes.

Desde el punto de vista metodológico, el proceso de diseño del corpus puso de manifiesto una trampa habitual en evaluación: un corpus demasiado fácil produce métricas perfectas que carecen de valor, mientras que uno demasiado difícil produce métricas tan bajas que tampoco son informativamente útiles. El equilibrio requiere modelar explícitamente los casos difíciles del dominio real.

6.2 Expectativa inicial versus resultados reales

El diseño original preveía tres niveles: el N1 (regex + entropía), un N2 basado en CodeBERT ajustado (*fine-tuned*) sobre FPSecretBench, y opcionalmente un N3 con Qwen 2.5 Coder 7B como validador contextual local. Los resultados obtenidos con Random Forest sobre características manuales son competitivos con lo que la literatura reporta para encoders ajustados en conjuntos de datos similares, lo que sugiere que la ventaja del N2 sobre el N1 se debe principalmente a las características contextuales diseñadas —no al modelo en sí. La hipótesis de que un encoder de código aportaría mejoras sustanciales sobre un clasificador clásico bien diseñado merece ser verificada con datos reales de mayor escala.

6.3 Mayores retos

El principal obstáculo técnico fue la indisponibilidad de SecretBench, que condicionó toda la estrategia de evaluación. El corpus propio, aunque diseñado con cuidado, no puede replicar la variedad de contextos de 818 repositorios reales. El segundo reto fue el diseño del corpus para evitar la separabilidad trivial: las dos primeras versiones producían resultados que, aunque numéricamente atractivos, no tenían valor experimental.

6.4 Trabajo futuro

Tres líneas de extensión resultan naturales. La primera y más urgente es la evaluación sobre SecretBench/FPSecretBench cuando el acceso esté disponible: los resultados actuales necesitan validación sobre datos reales de mayor escala para confirmar su generalización. La segunda es sustituir el Random Forest por CodeBERT (*fine-tuned*) sobre las características contextuales extraídas, lo que permitiría comparar directamente la ventaja de un encoder de código frente a características manuales. La tercera es añadir al catálogo los tipos de secreto no cubiertos detectados durante la validación real: credenciales en formato de fichero de configuración sin comillas y URLs con autenticación *basic*.

6.5 Limitaciones del trabajo

Los resultados presentados son válidos bajo las siguientes condiciones, que el lector debe considerar:

- El corpus de evaluación es sintético. Aunque diseñado con cuidado, no garantiza la distribución de tipos de secreto y falsos positivos que se encontraría en producción real a escala.
- Las dos características más informativas del N2 (`ctx_in_test_file`, `ctx_sample_comment`) dependen de convenciones de nomenclatura y comentarios que no son universales. En proyectos que no siguen estas convenciones, el rendimiento puede degradarse.
- La comparativa con GitLeaks y TruffleHog en modo *filesystem* no refleja su caso de uso habitual, que incluye análisis del historial git. En ese modo, ambas herramientas pueden detectar secretos ya eliminados del árbol de trabajo, capacidad que el sistema actual no tiene.
- El sistema no valida si una credencial detectada sigue activa, lo que puede generar ruido operativo en repositorios con rotación frecuente de credenciales.

Bibliografía

- Basak, S. K., Neil, L., Reaves, B., & Williams, L. (2023). SecretBench: A Dataset of Software Secrets. *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. <https://arxiv.org/abs/2303.06729>
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., & Zhou, M. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Languages. *Findings of the Association for Computational Linguistics: EMNLP 2020*.
- GitGuardian. (2024). *The State of Secrets Sprawl 2024*. GitGuardian. <https://www.gitguardian.com/state-of-secrets-sprawl-report-2024>
- IBM Security. (2024). *Cost of a Data Breach Report 2024*. IBM Corporation.
- Jiménez Ormad, R. (2026). *Detector de secretos en repositorios Git: Módulo 6 Tarea 2*. <https://github.com/Ruben1386/secret-scanner-master-m6->
- Leijssen, Z., & Gitleaks Contributors. (2024). *Gitleaks: Protect and Discover Secrets Using Gitleaks*. <https://github.com/gitleaks/gitleaks>
- NIST. (2022). *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities* (NIST Special Publication 800-218). National Institute of Standards; Technology.
- Rahman, M. N., Ahmed, S., Wahab, Z., Sohan, S. M., & Shahriyar, R. (2025). Secret Breach Detection in Source Code with Large Language Models. *arXiv preprint arXiv:2504.18784*.
- Saha, A., Denning, T., Srikumar, V., & Kasera, S. K. (2020). Secrets in Source Code: Reducing False Positives Using Machine Learning. *2020 International Conference on COMMunication Systems & NETWORKS (COMSNETS)*, 168-175.
- Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal*, 27(3), 379-423.
- Trufflesecurity. (2024). *TruffleHog: Find, Verify, and Analyze Leaked Credentials*. <https://github.com/trufflesecurity/trufflehog>

Anexos

Anexo A — Estructura del repositorio de código

Repositorio público: <https://github.com/Ruben1386/secret-scanner-tfm>

```
secret-scanner-tfm/
├── src/
│   ├── corpus_generator.py # Generador de corpus reproducible (--seed 42)
│   ├── tier1_baseline.py   # Detector N1: regex + entropía + extracción features
│   ├── tier2_classifier.py  # Clasificador N2: Random Forest
│   ├── evaluate_tools.py   # Comparativa vs GitLeaks y TruffleHog
│   ├── validate_real_repos.py # Validación sobre repos reales
│   └── generate_figures.py  # Generación de figuras para el documento
├── results/
│   ├── tier1_metrics.json  # Métricas del N1 sobre corpus completo
│   ├── tier2_metrics.json  # Métricas del N2 (CV + held-out)
│   ├── comparison_table.json # Tabla comparativa 4 sistemas
│   ├── real_world_validation.json # Resultados validación real
│   └── figures/            # Figuras PNG 300 dpi para el documento
├── doc/                   # Este documento (pdf/html/md), slides y ficheros de soporte
├── .gitignore
├── LICENSE                # MIT License
└── README.md
```

Los ficheros generados por los scripts (`data/corpus/`, `results/tier1_candidates.jsonl`, `results/tier2_model.joblib`) no se versionan: se reproducen en segundos siguiendo el protocolo del Anexo C. `data/corpus/` en particular contiene credenciales sintéticas con formato realista que el escaneo de secretos de GitHub bloquea por *Push Protection*.

Anexo B — Catálogo de patrones del Nivel 1

Catálogo completo de patrones del Nivel 1.

#	Tipo de secreto	Ejemplo de formato detectado
1	AWS Access Key ID	AKIA[A-Z0-9]{16}
2	AWS Secret Access Key	aws_secret_access_key = "..."
3	GitHub PAT	ghp_[A-Za-z0-9]{36}
4	GitHub OAuth Token	gho_[A-Za-z0-9]{36}
5	GitHub App Token	ghu_/ghs_[A-Za-z0-9]{36}
6	GitHub Fine-grained PAT	github_pat_[A-Za-z0-9]{82}
7	Google API Key	AIza[A-Za-z0-9_-]{35}
8	Slack Token	xox[baprs]-[A-Za-z0-9-]{10,}
9	Stripe Live Secret Key	sk_live_[A-Za-z0-9]{24,}
10	Stripe Test Secret Key	sk_test_[A-Za-z0-9]{24,}
11	OpenAI API Key	sk-[A-Za-z0-9_-]{20,}
12	Anthropic API Key	sk-ant-api03-[A-Za-z0-9_-]{20,}
13	JWT Token	eyJ...eyJ...<sig>
14	Clave privada (RSA/SSH/EC/PGP)	-----BEGIN ... PRIVATE KEY-----
15	Asignación genérica de contraseña	password = "..."
16	Asignación genérica de API key	api_key = "..."
17	Asignación genérica de secreto	secret = "..."
18	Asignación genérica de token	access_token = "..."
19	Cadena de conexión a base de datos	postgresql:// user:pass@host
20	Entropía Shannon $\geq 4,5$	Cualquier token ≥ 20 chars entre comillas

Anexo C — Protocolo de reproducción de resultados

```
# 1. Clonar el repositorio
git clone https://github.com/Ruben1386/secret-scanner-tfm.git
cd secret-scanner-tfm

# 2. Crear entorno virtual
```

```
python3.12 -m venv .venv && source .venv/bin/activate

# 3. Instalar dependencias
pip install pandas numpy scikit-learn matplotlib seaborn joblib

# 4. Generar corpus (reproducible)
python src/corpus_generator.py --out data/corpus --seed 42

# 5. Ejecutar Nivel 1
python src/tier1_baseline.py --corpus data/corpus --out results/

# 6. Entrenar y evaluar Nivel 2
python src/tier2_classifier.py --candidates results/tier1_candidates.jsonl --out results/

# 7. Generar figuras
python src/generate_figures.py --results results/ --out results/figures/
```

Tiempo estimado de ejecución completa: < 2 minutos en CPU de 4 núcleos.